

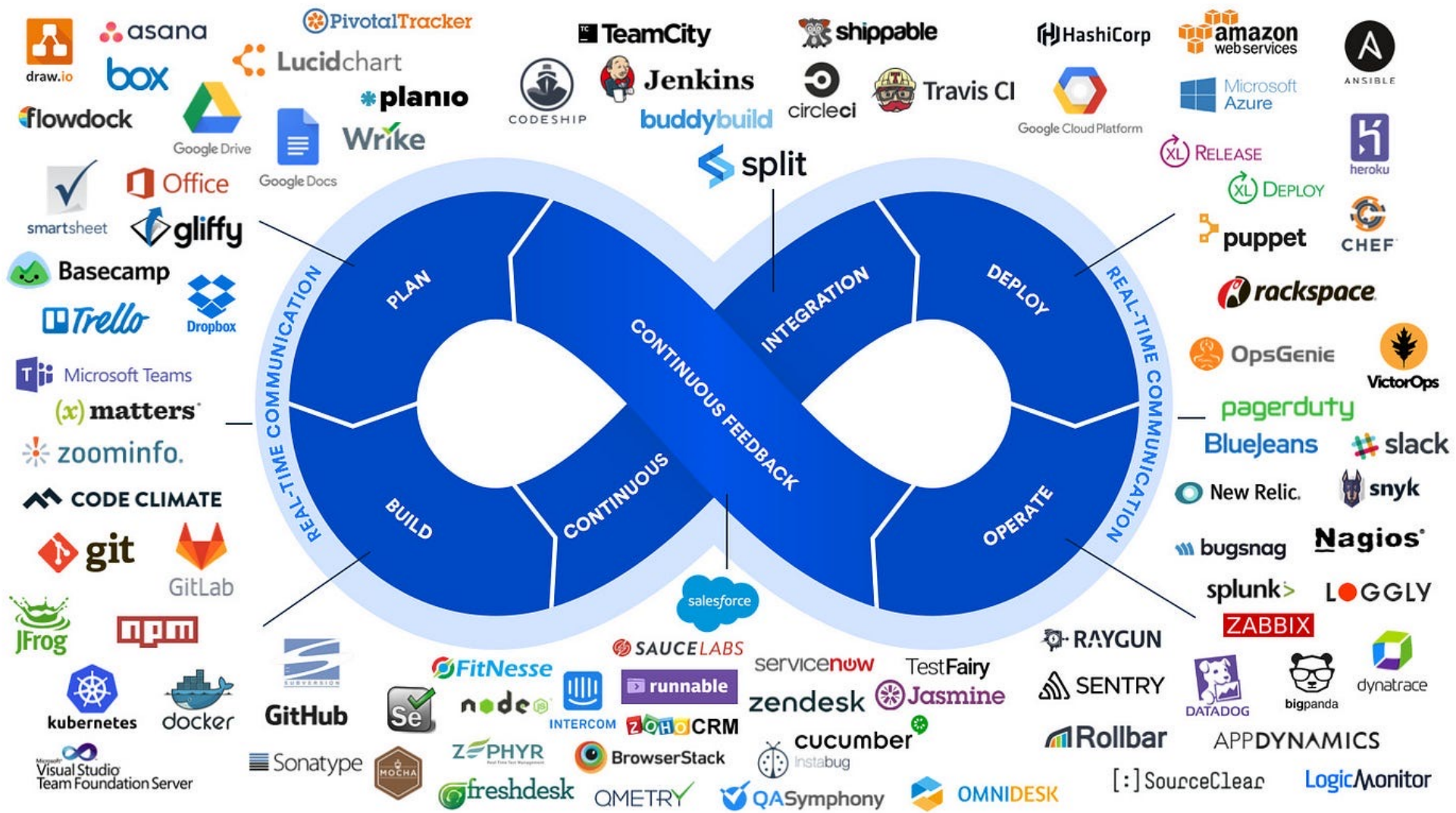
Program Roadmap Ahead

DevOps and Cloud Labs

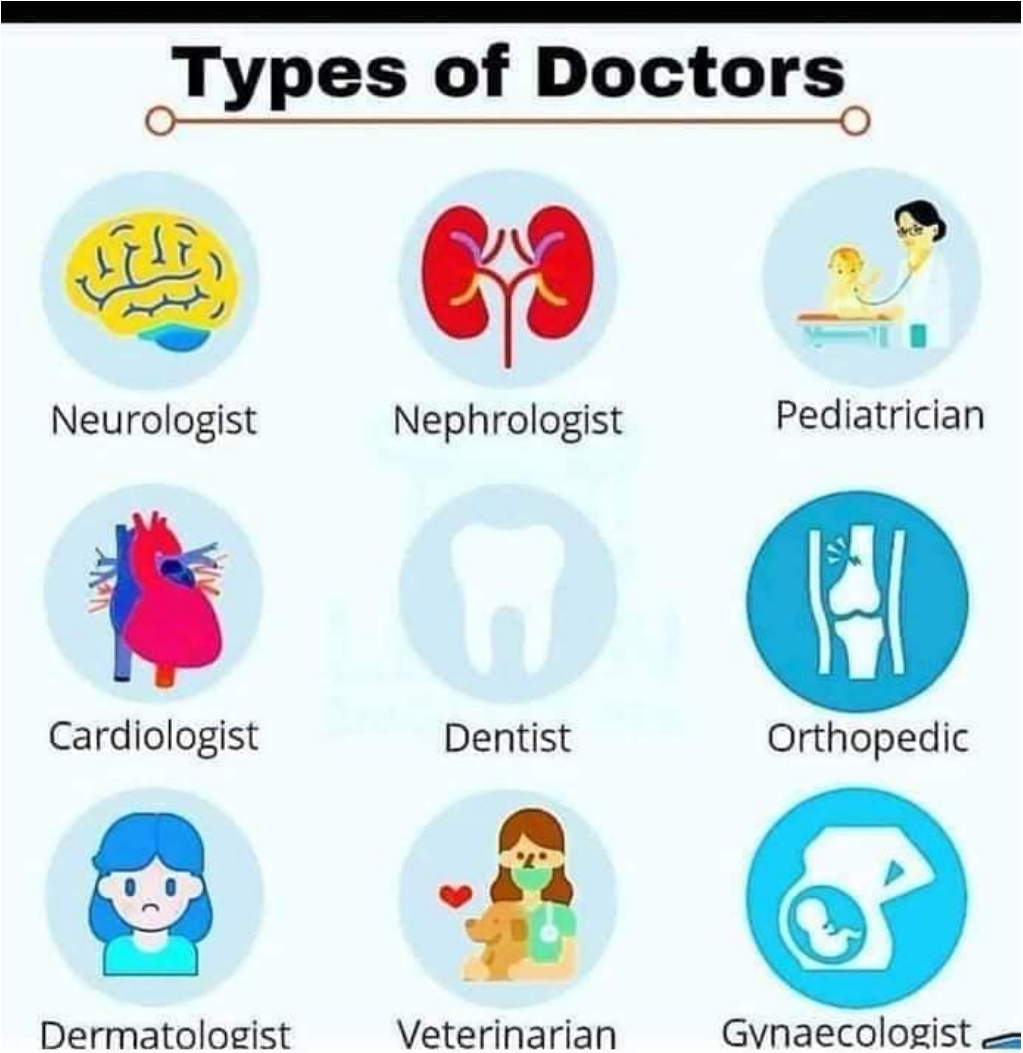
<https://topmate.io/cloud>

<https://www.youtube.com/@DevOps-Cloud>

So many tools, technologies and process



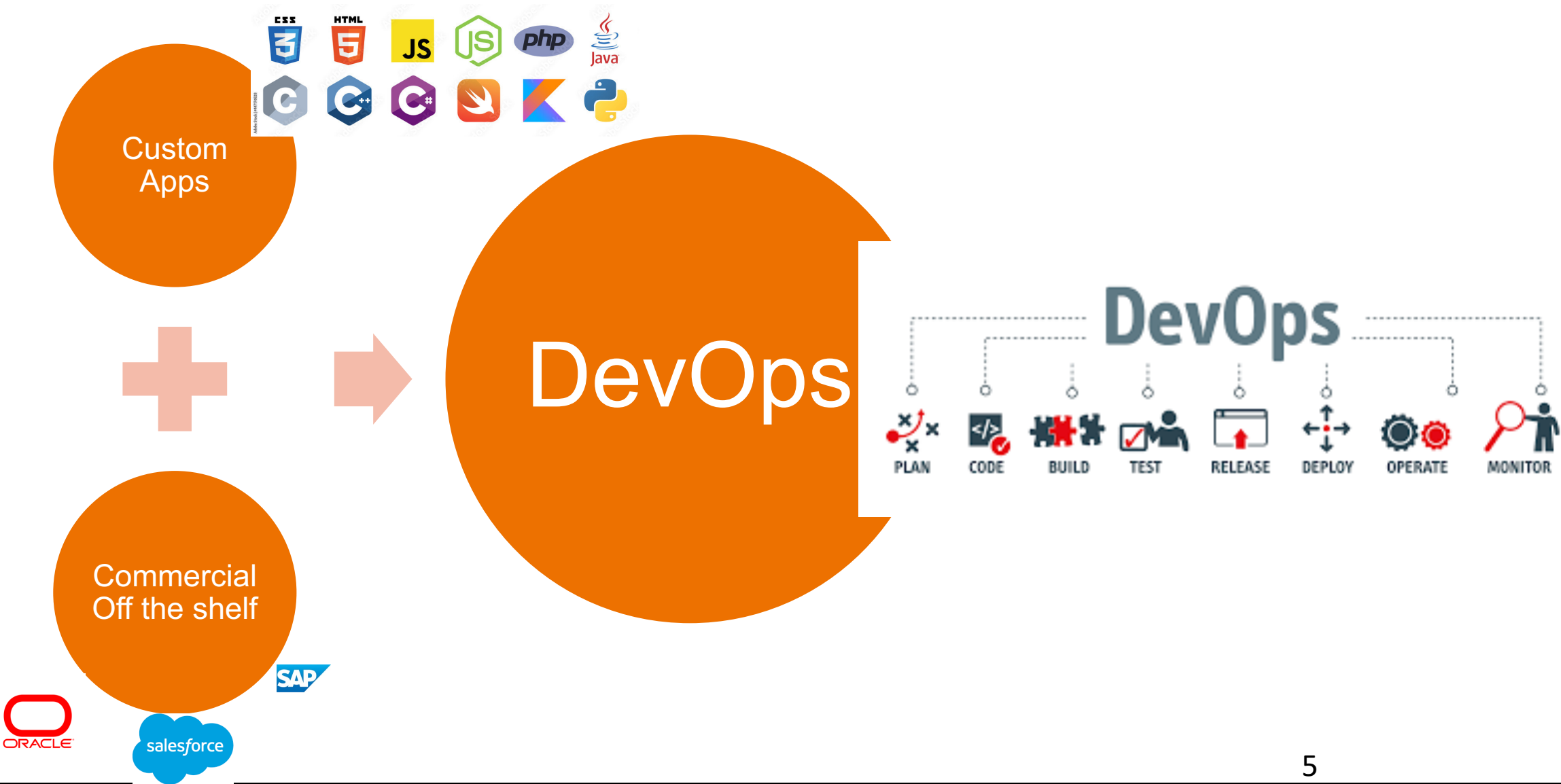
DevOps Expert needs to know all



DevOps Mindset



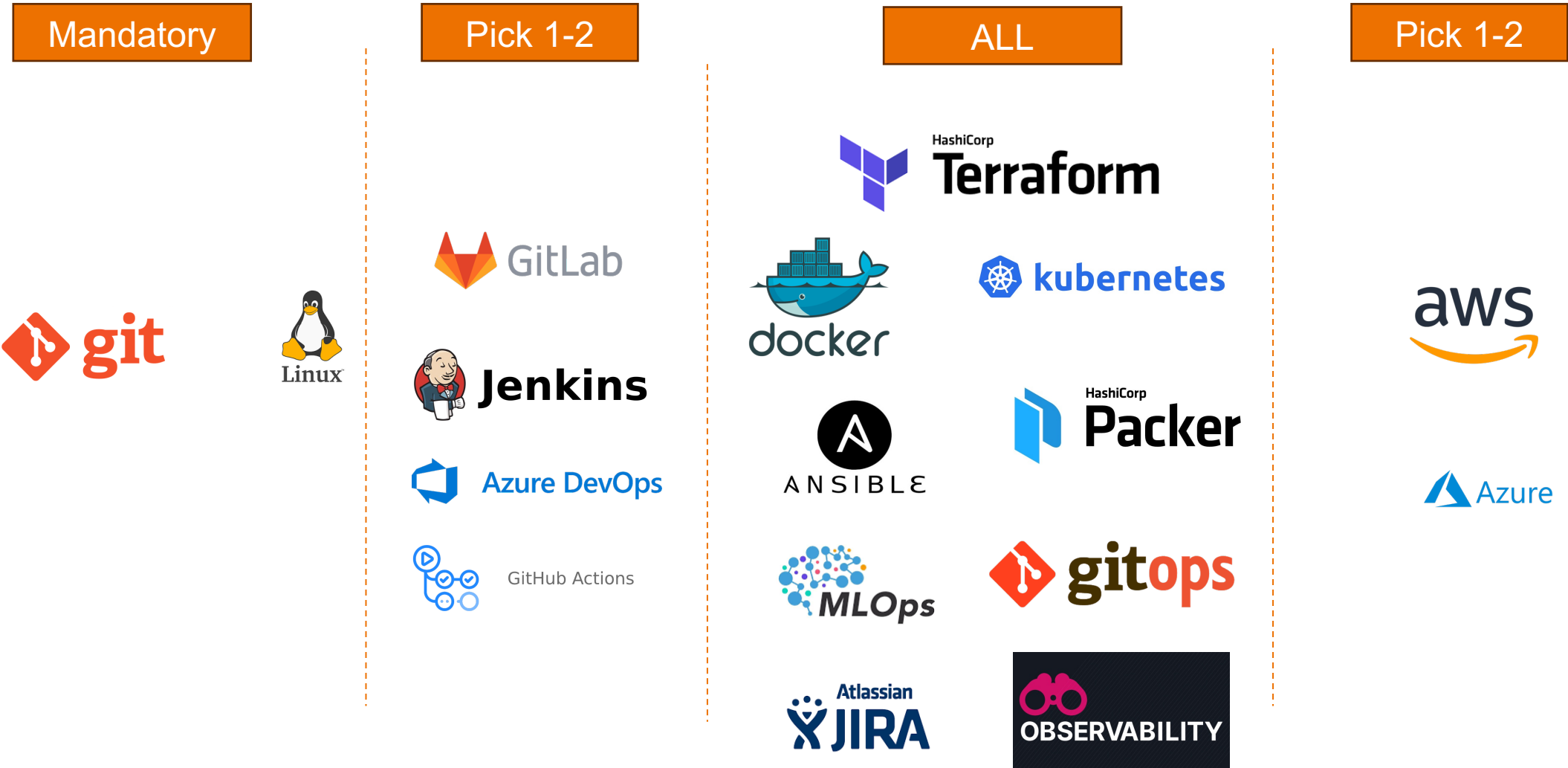
Application Types



Key capabilities to learn

1. Version Control Systems (VCS) and Operating Systems - Basics
2. Continuous Integration (CI)
3. Infrastructure as Code (IaC) - Infrastructure Provisioning
4. Configuration Management (CM)
5. Containerization and Orchestration – Docker and Kubernetes
6. Continuous Deployment (CD) – Deployment strategies, release automation
7. Cloud Platforms – AWS, Azure, GCP
8. Operations in Cloud –
 - Scalability and High Availability / Disaster Recovery and Backup
 - Monitoring, Logging, Observability and Alerting
 - Security and Compliance (DevSecOps)
 - Cost Optimization

Roadmap for DevOps and Cloud Learning and we cover them ALL



Roadmap Ahead (Live Sessions)

DevOps:

Git, Linux

Build and CICD,

Docker and Kubernetes

Terraform and Ansible

AWS Cloud – 6 to 8 Sessions

4 Projects (Those who are ready can start this)

Who will benefit the most?

Attend on regular basis – 15% benefit

Work on assignments – 35% benefit

Working on Projects – 40% benefit

Demo – 10% benefit

Recordings of previous Batch Sessions

Available PDF in the #resources section in Slack...

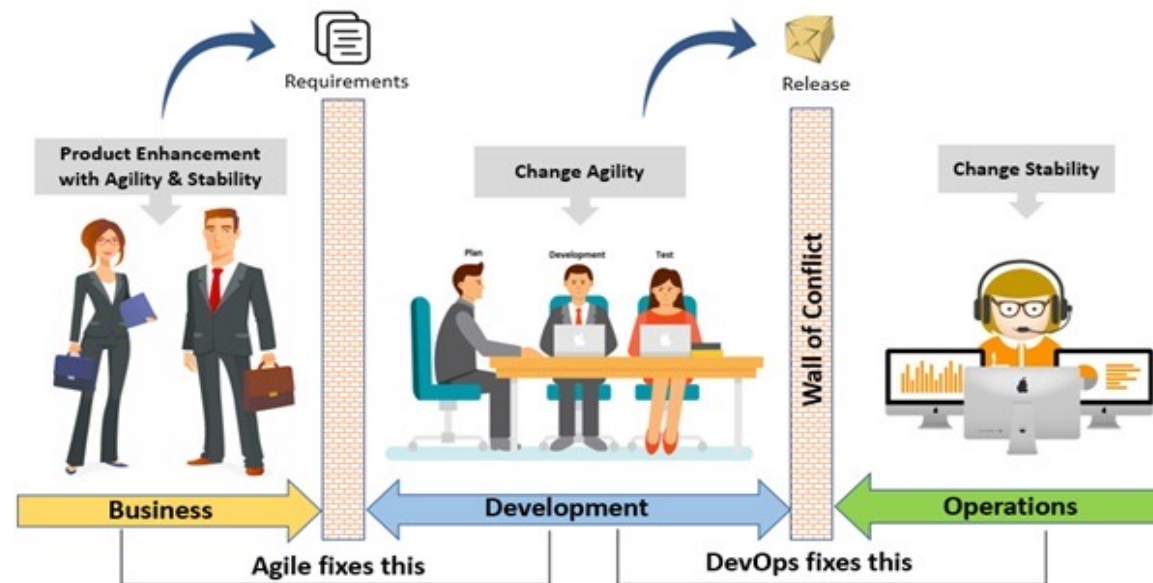
Rewards...For every professionally done Project demo



What is DevOps ?

DevOps is a set of practices, tools, and a cultural philosophy that automate and integrate the processes between software development and IT teams.

It emphasizes team empowerment, cross-team communication and collaboration, and technology automation.



The word DevOps is a combination of two words that is development and operations.

This is a culture that promotes the collaboration between development team and the operations team. With the help of DevOps, the applications can be delivered faster and serve their customer needs nicely.

Dev Challenges

How DevOps addresses Dev Challenges .



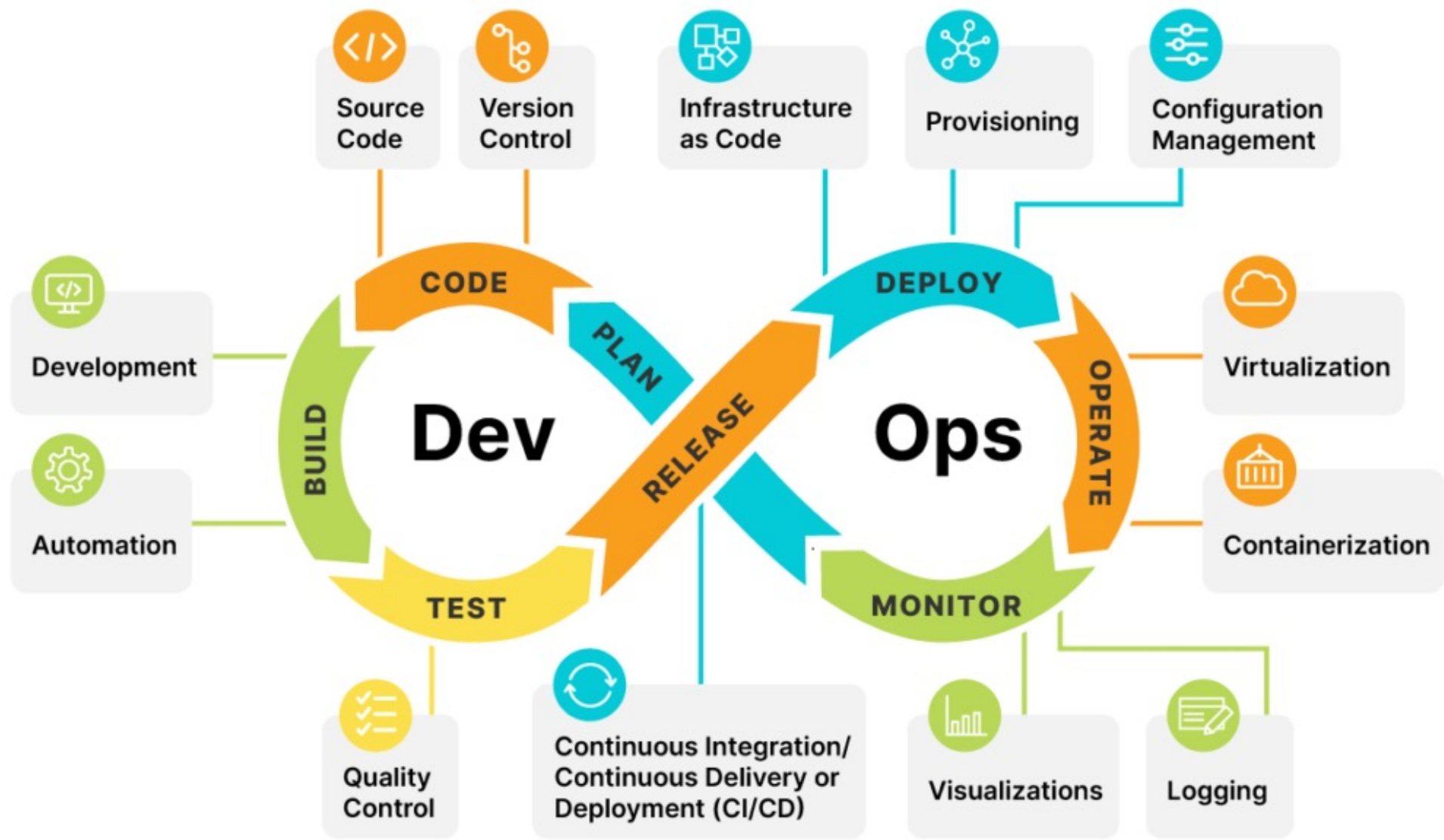
Developers	Dev Challenges	DevOps Solutions
	Code deployment time is higher	Quick Deployment of Code
	Poor code management	Effective code management due to continuous integration

Ops Challenges

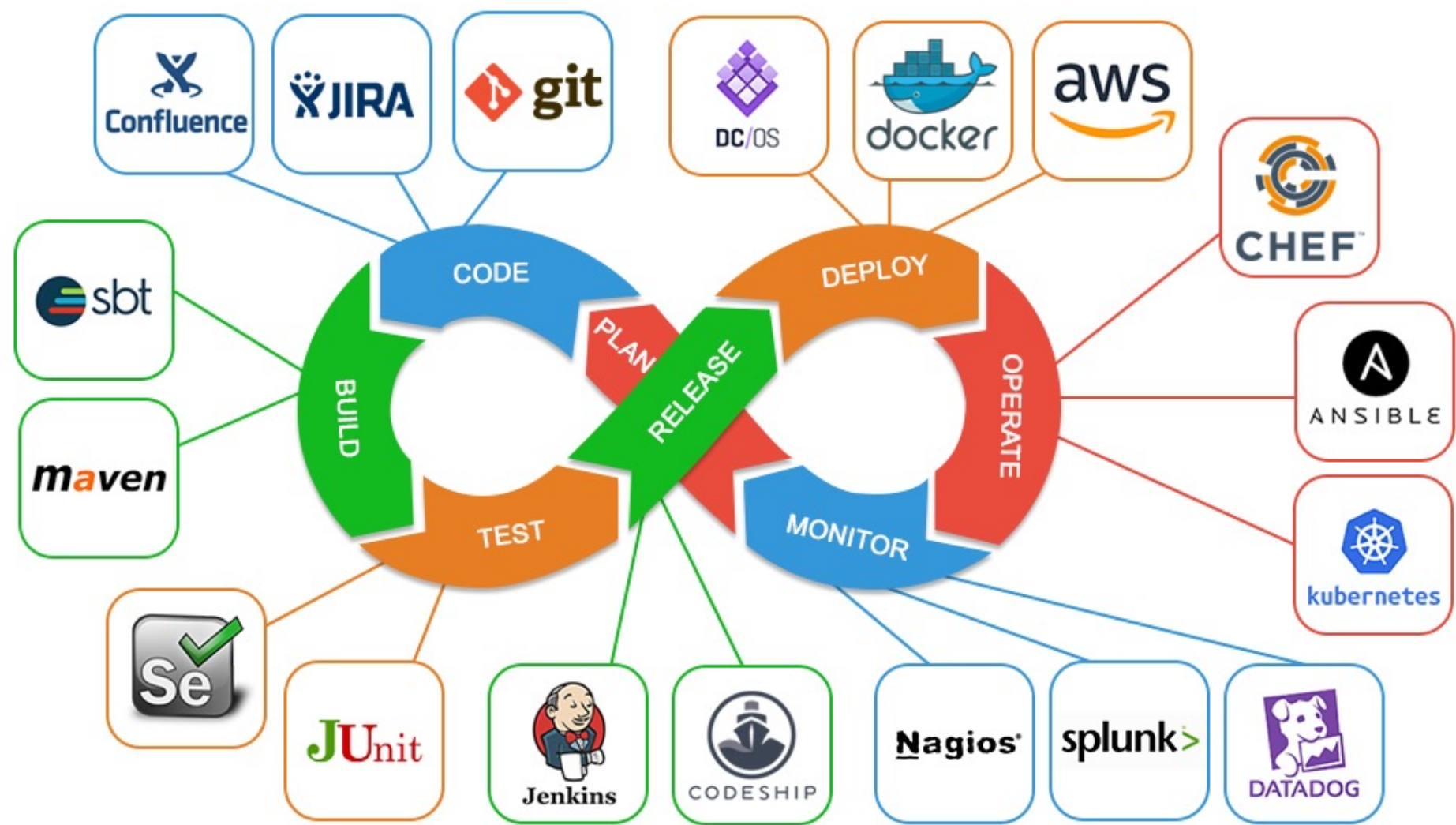
How DevOps addresses Ops Challenges .

Operations	Ops Challenges	DevOps Solutions
	Uptime maintenance is difficult	Maximum reliability of Software Uptime
	Automation tools are not so effective	Automation tools are highly effective
	Server monitoring is tough	Continuous Monitoring
	Feedback is not given well	Continuous Monitoring

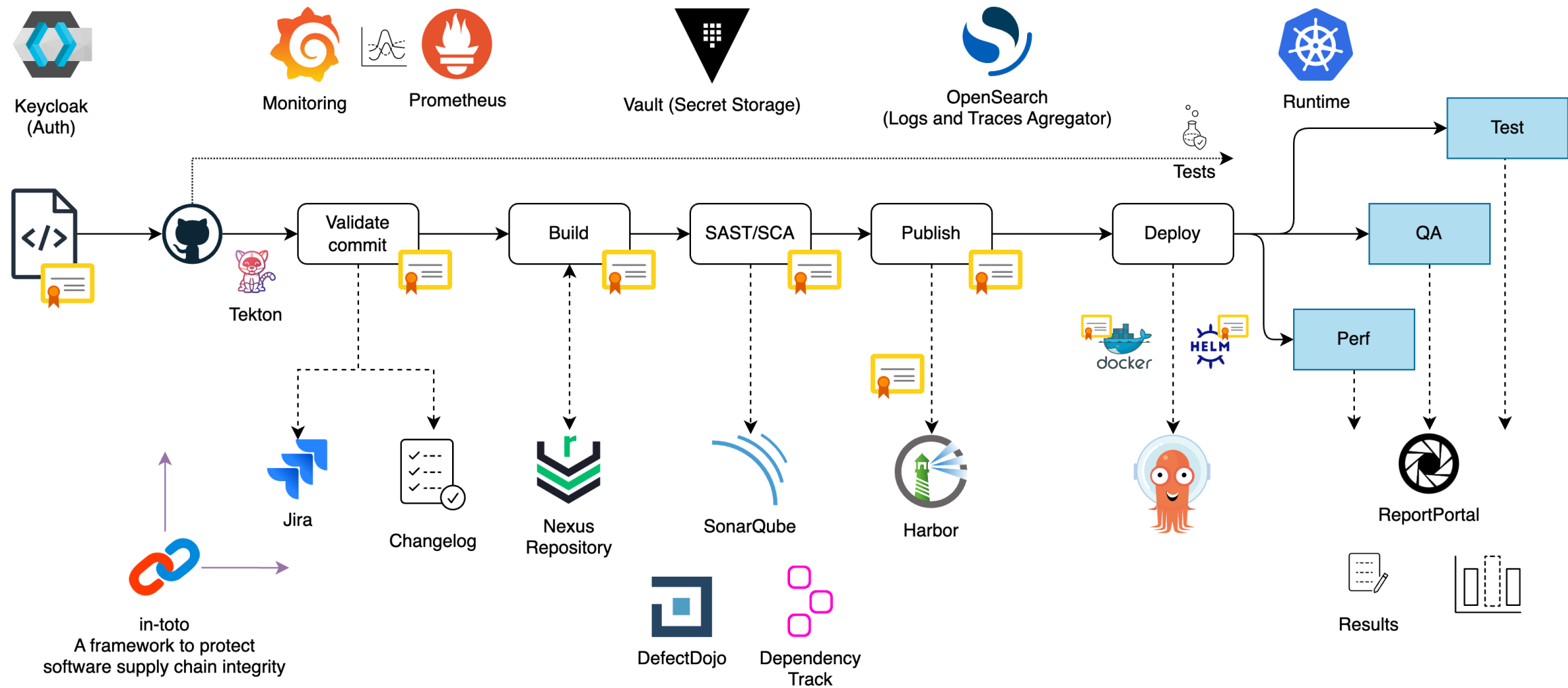
DevOps Lifecycle



DevOps - Tools view



Typical DevOps Workflow



DevOps tools



Build

SCM & VCS

git, Bitbucket, GitHub, mercurial

Continuous Integration

circleci, Jenkins, Bamboo, Travis CI, CODESHIP, TeamCity

Build

sbt, Gradle, Maven, MSBuild

Repositories

Jfrog Artifactory, Nexus, npm, docker hub, Amazon S3



Test

Browser

Cypress, Karma, Selenium, Calibre Framework, Browsersync

Load / Performance

JMeter, SOASTA, BlazeMeter, Locust, Gatling, Load Impact



Deploy

Configuration Management

puppet, CHEF, SALTSTACK, ANSIBLE, PowerShell, CFEngine

Continuous Delivery

urban (code), Xebia Labs, harness, Automtic, Spinnaker, go, Octopus Deploy, Electric Cloud



Infrastructure Provisioning

Security

HackCorp Vault



Run

Containers

docker, rkt

Orchestration

RANCHER, Nomad, Kubernetes, MESOS, AWS, OpenShift

Compute

Microsoft Azure, AWS, Google Cloud Platform

Serverless

AWS Lambda, Azure Functions

Run-Time

java, python, Ruby, node.js, .NET

OS

ubuntu, Core OS, redhat

DB

MySQL, mongoDB, PostgreSQL, Amazon RDS, Amazon ElastiCache

CDN

Akamai, fastly, Cloudflare, NetScout Systems



Monitor

Time-Series Metrics

Synthetic / Availability: pingdom, Apica, catchpoint, gomez, keynote

Tracing: LIGHTSTEP, ELK, ZIPKIN

Big Data: unravel, Anodot

Application (APM): AWS X-Ray, INSTANA, dynatrace, New Relic, APPDYNAMICS

Cloud Infrastructure: StatsD, DATADOG, wavefront, signal fx, Prometheus, Grafana, honeycomb, graphite, CloudWatch

Traditional Infrastructure: Nagios, zenoss

Network: ThousandEyes, ExtraHop, LogMonitor, solarwinds

Unstructured Data

Errors: SENTRY, Rollbar, Stackify, Airbrake.io, OverOps, RAYGUN

Logs Mgmt: splunk, sumologic, LOGGLY, logz.io, elastic, kibana, logstash, SCALYR



Manage

Events & Alerts

Moogsoft, signifai, servicenow, bigpanda

Service Desk

servicenow, bmc, JIRA

Project Management

asana, JIRA, PivotalTracker, Trello

CRM Helpdesk

zendesk, freshdesk



Notify

Escalate

pagerduty, matters, everbridge, VictorOps, OpsGenie

Collaboration

slack, HipChat, Microsoft Teams, box, twilio

18

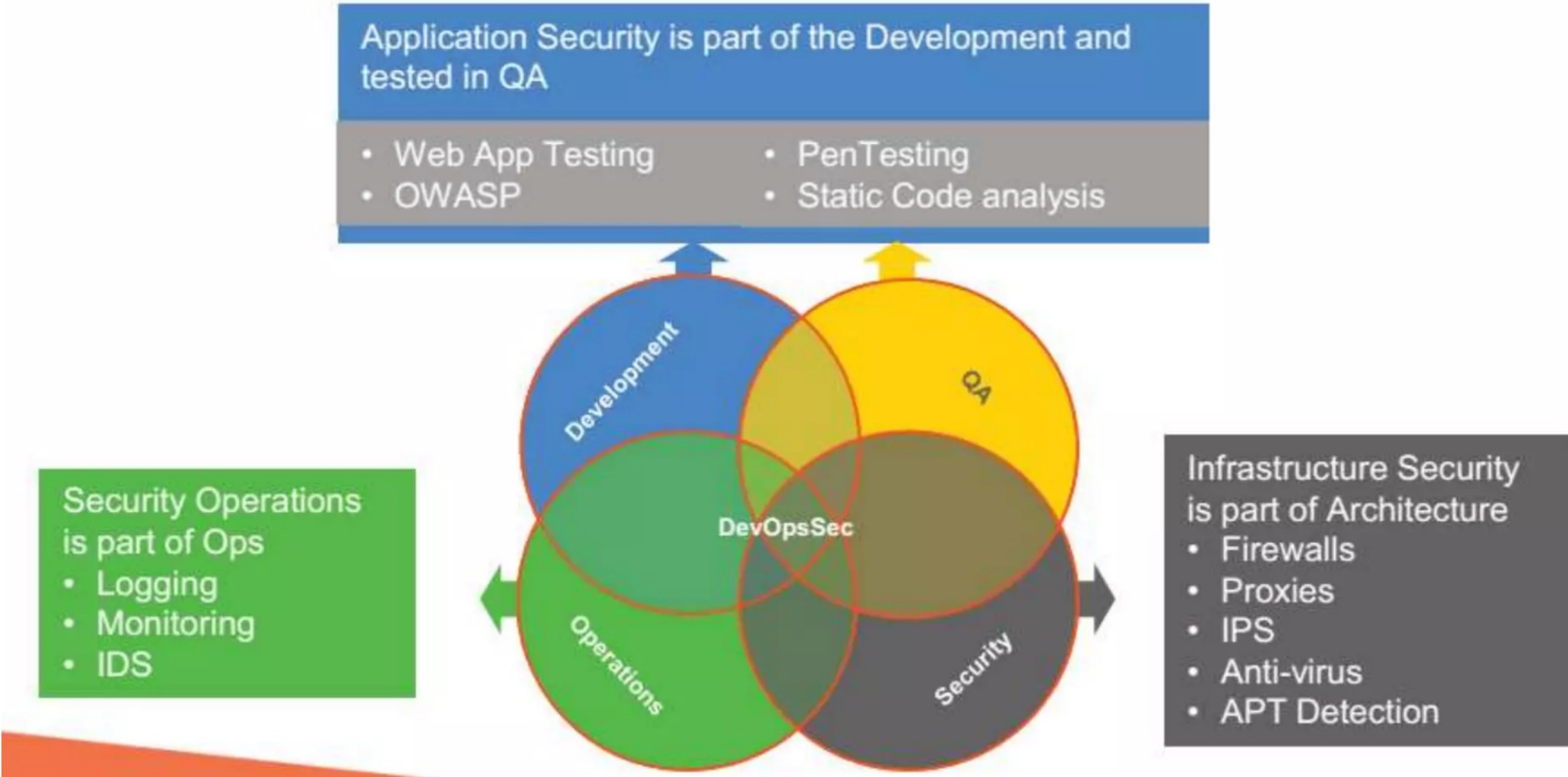
<https://topmate.io/cloud>

<https://www.youtube.com/@DevOps-Cloud>

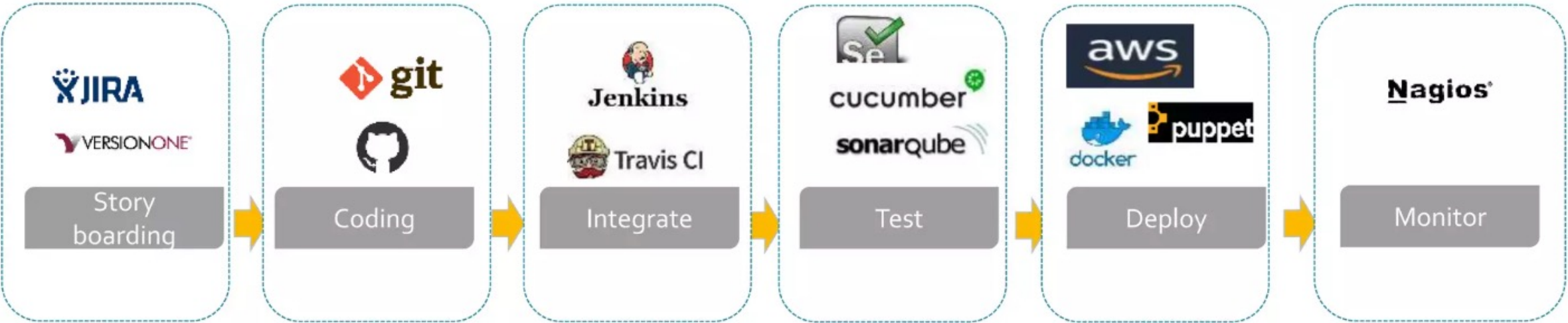
But someone said DevSecOps ?



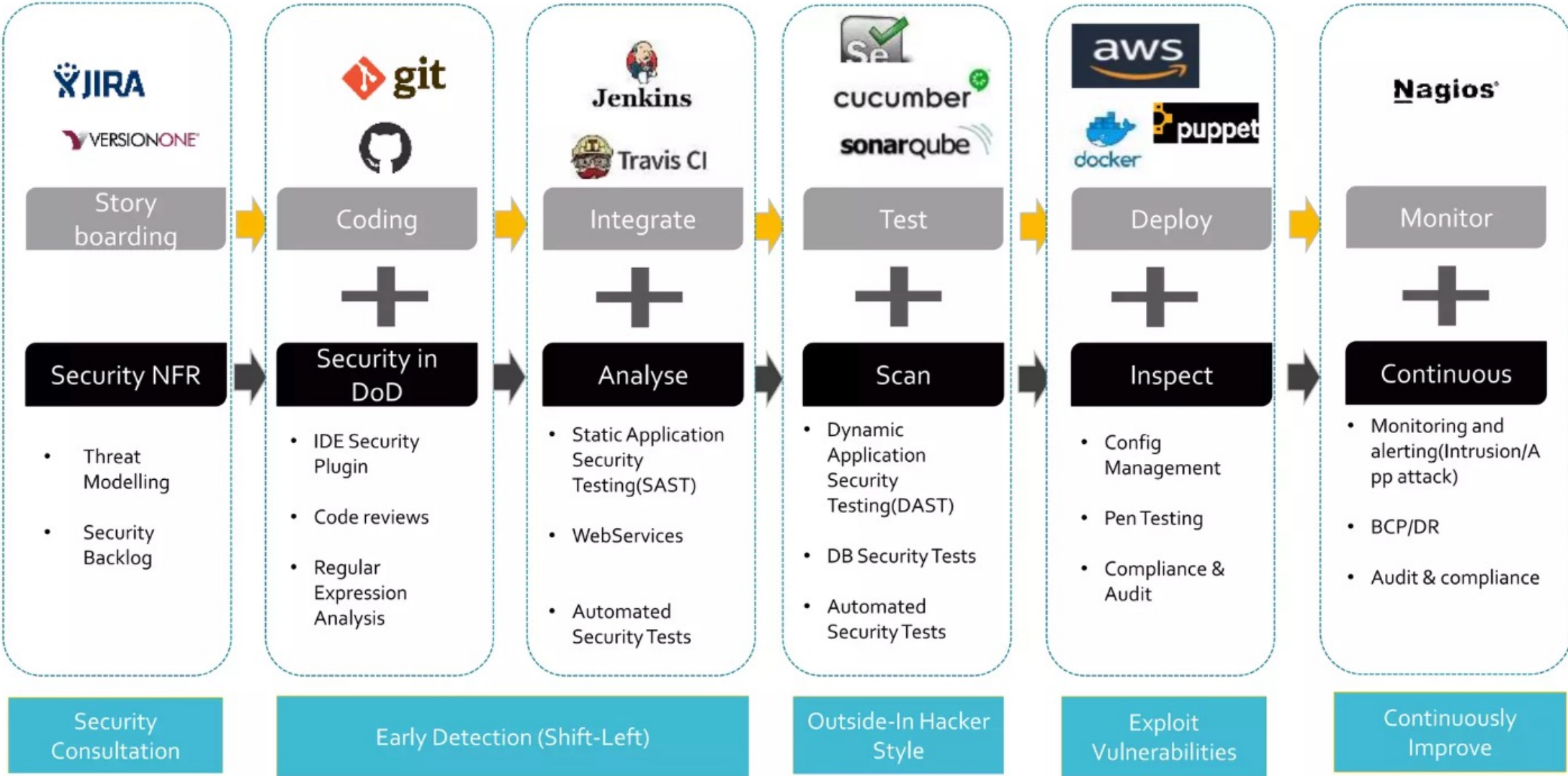
DevSecOps



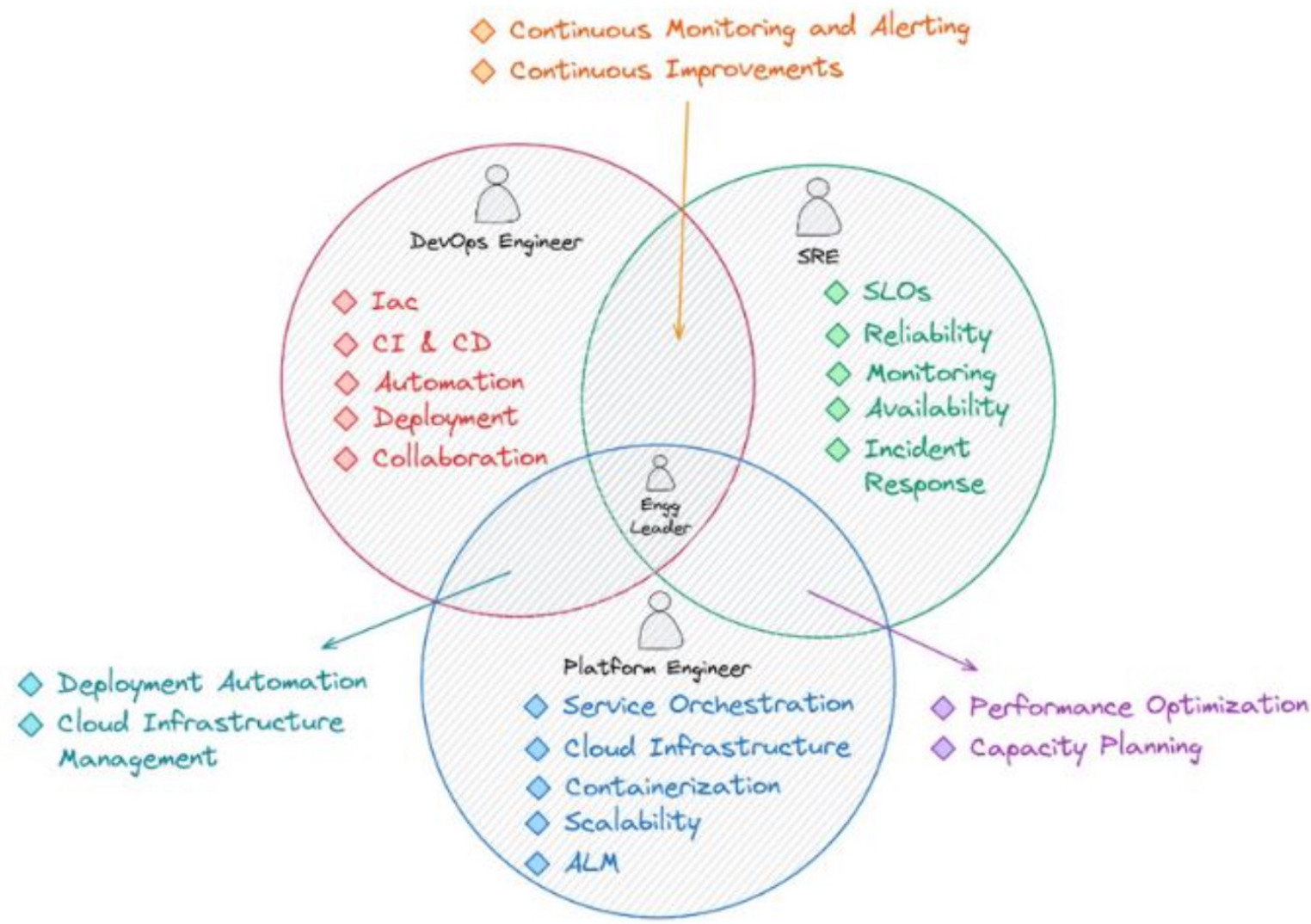
DevOps Pipeline



DevSecOps Pipeline



Role - DevOps Engineer / SRE / Platform Engineer

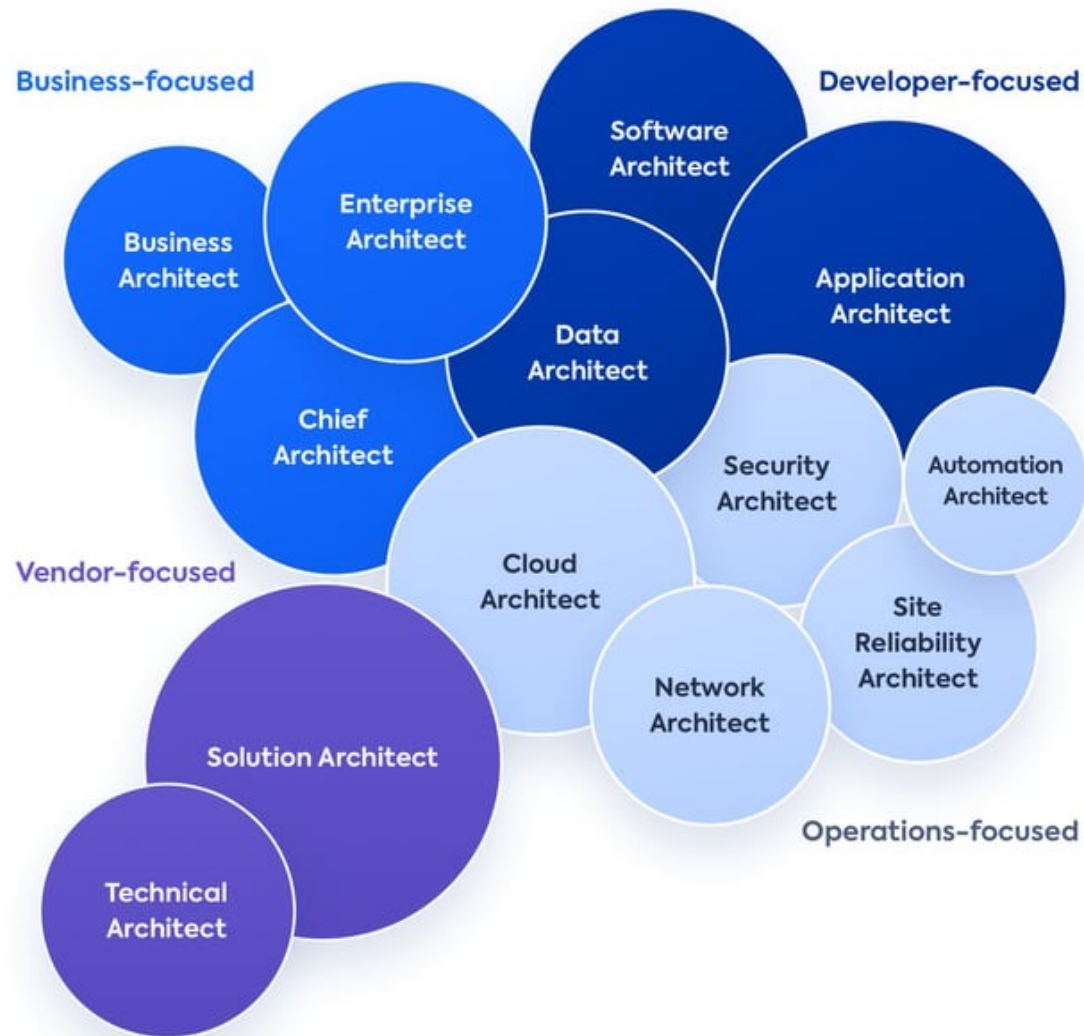


Role – Cloud Engineer

Cloud engineer skills at a glance



Role - Architect



Business-focused

The main focus of business-focused architects is to ensure that all IT systems are managed and maintained to meet the organization's goals.

Developer-focused

Developer-focused IT architects – such as software architects, application architects, and data architects work on system design from the programming and software development aspects.

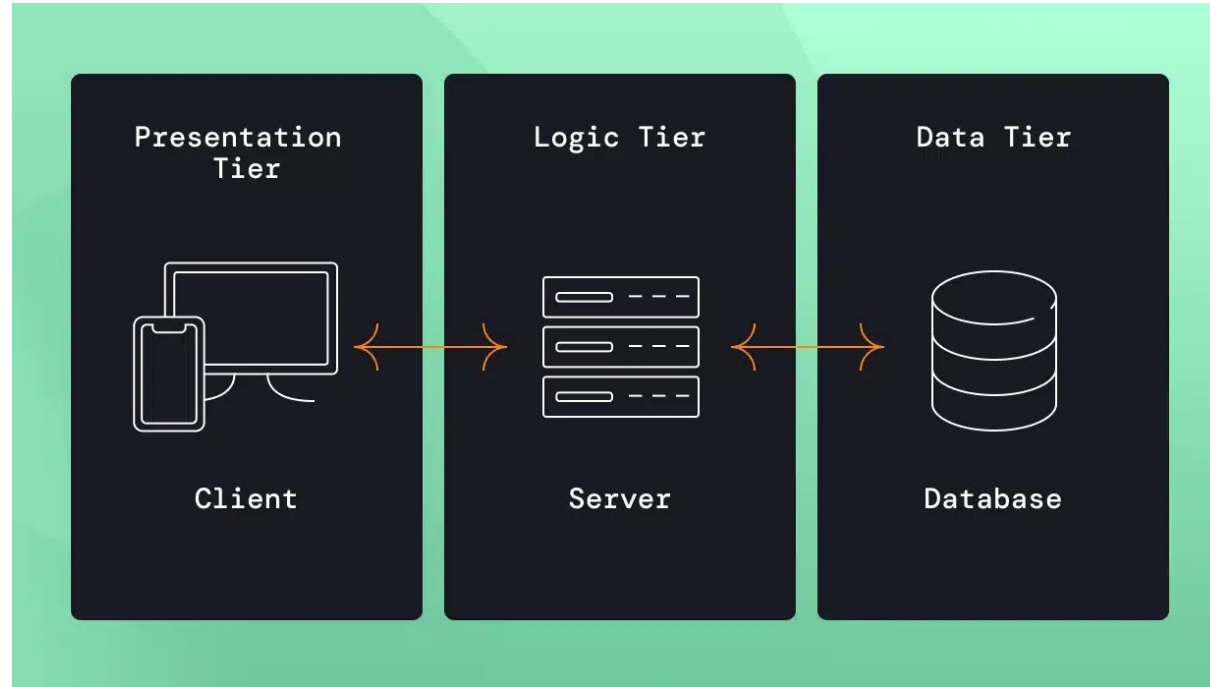
Operations-focused

Their role primarily deals with designing systems that are in line with the operational resources and capabilities of the organization's digital infrastructure.

Vendor-focused

Vendor-focused architects, such as solution architects and technical architects, apply a product or service to system designs intended to achieve business goals.

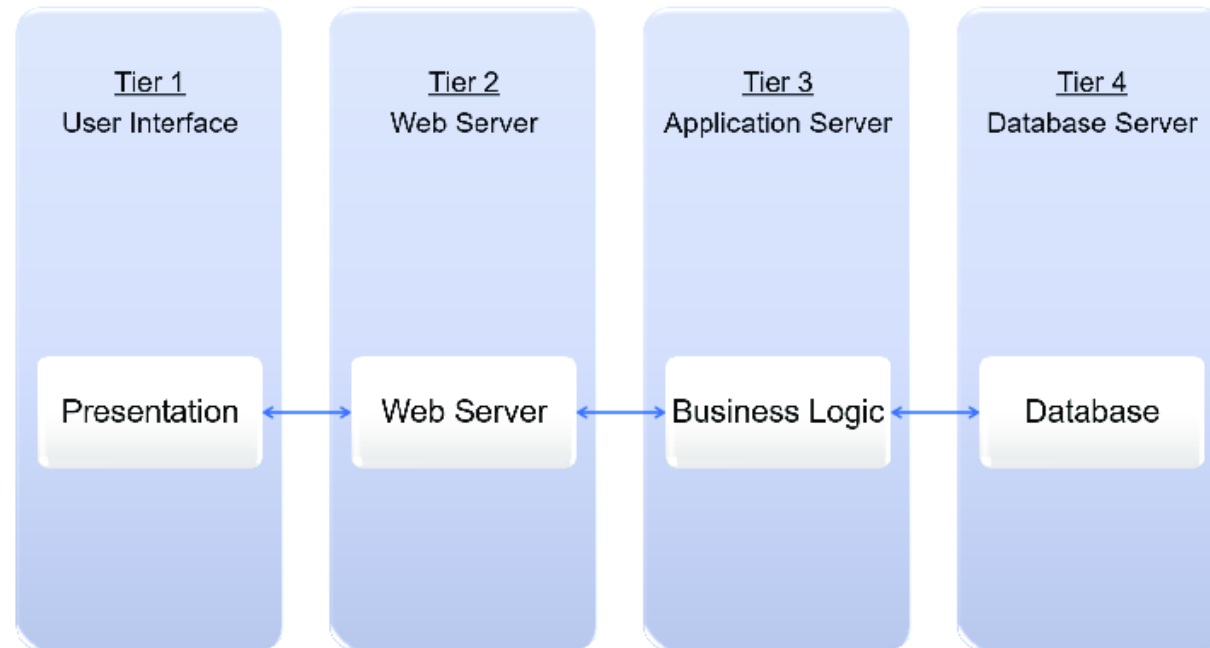
3 Tier Application Architecture



3-Tier Architecture

- **Presentation layer:** Typically handled by frontend developers.
- **Logic layer:** DevOps engineers might be involved in deploying and managing application servers, middleware, and APIs.
- **Data layer:** DevOps engineers might be involved in managing databases and data storage.

4 Tier Application Architecture



Presentation layer: Typically handled by frontend developers.

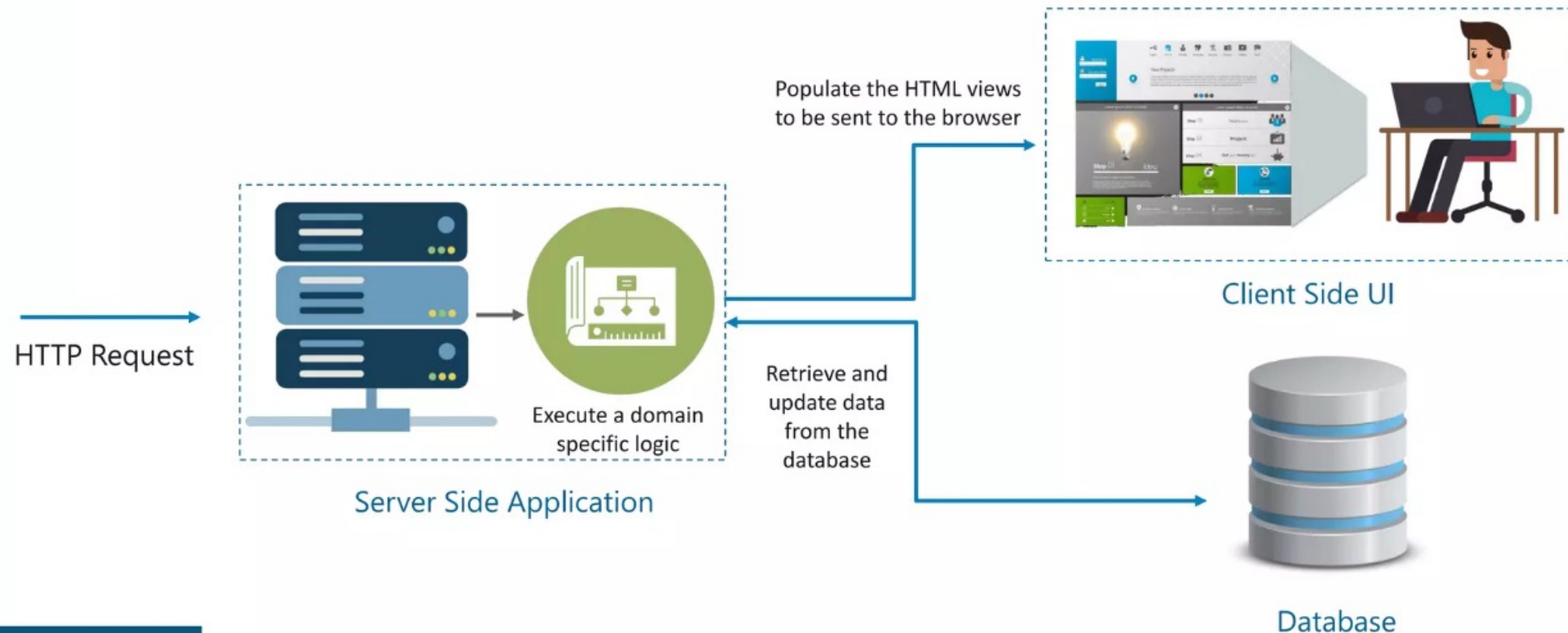
Web Server: DevOps engineers might have more responsibilities, including managing multiple application servers and load balancers.

Business Logic: DevOps engineers might be involved in managing message queues, service buses, and other integration components.

Data layer: DevOps engineers might be involved in managing databases and data storage.

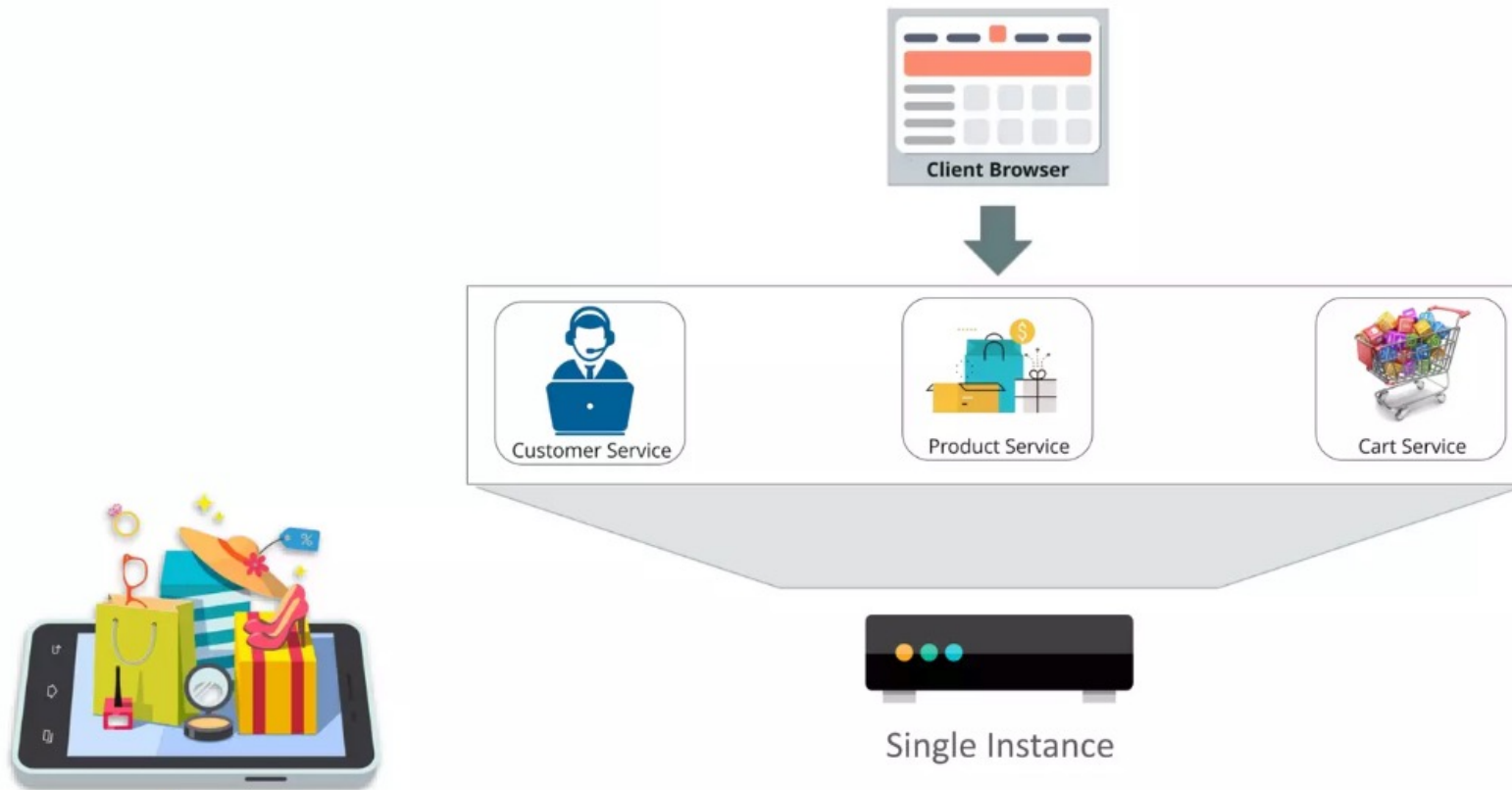
Before Microservices – Monolithic Architecture

Monolithic Architecture is like a big container wherein all the software components of an application are assembled together and tightly packaged

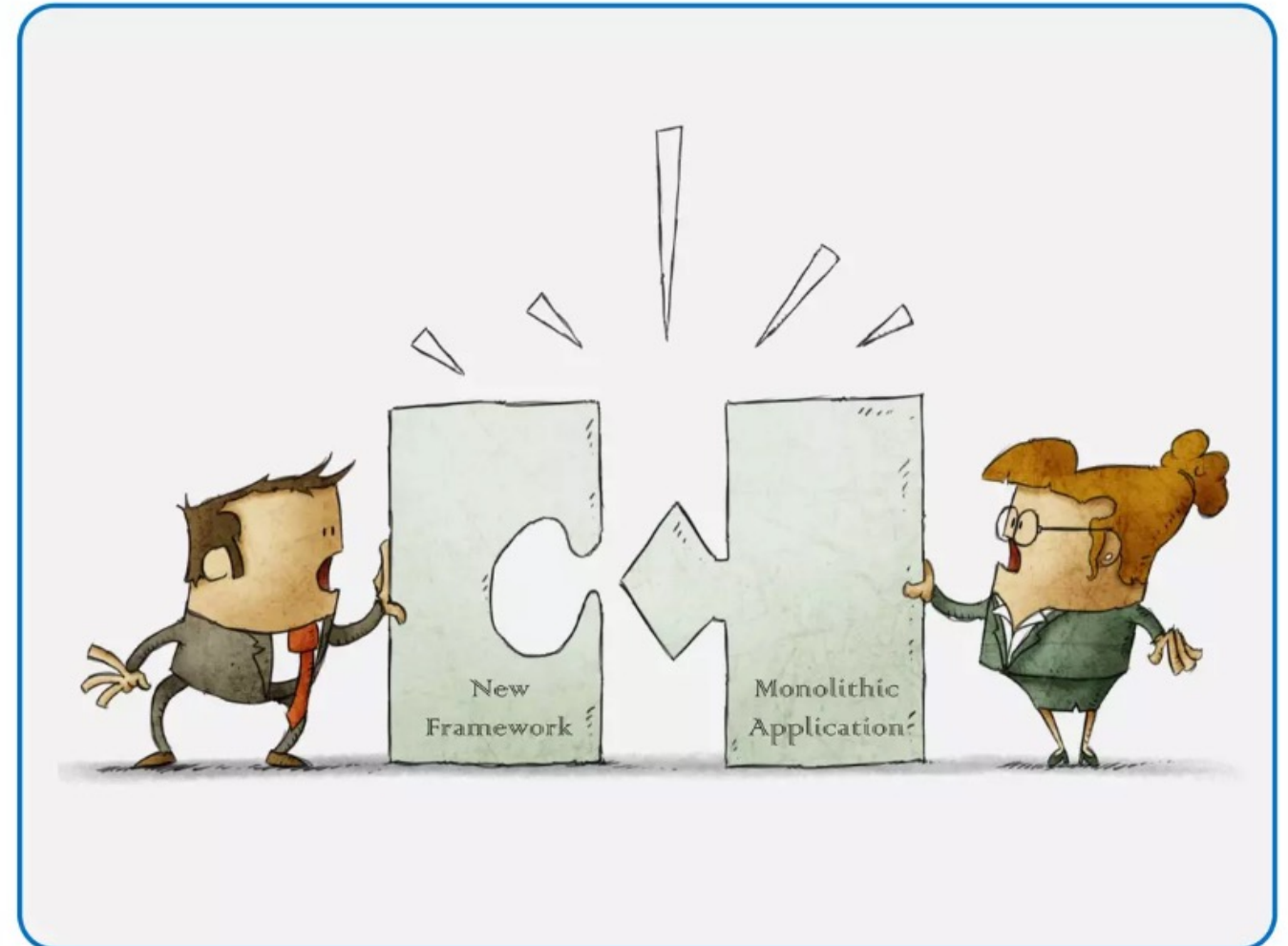


Monolithic Architecture - Example

Let's take a classic use case of an E-Commerce Application

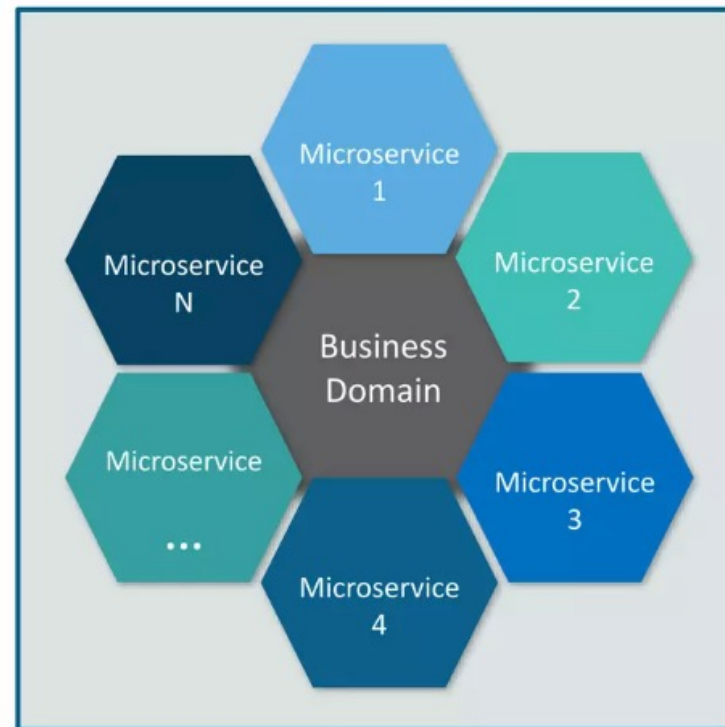


Monolithic Architecture - Challenges



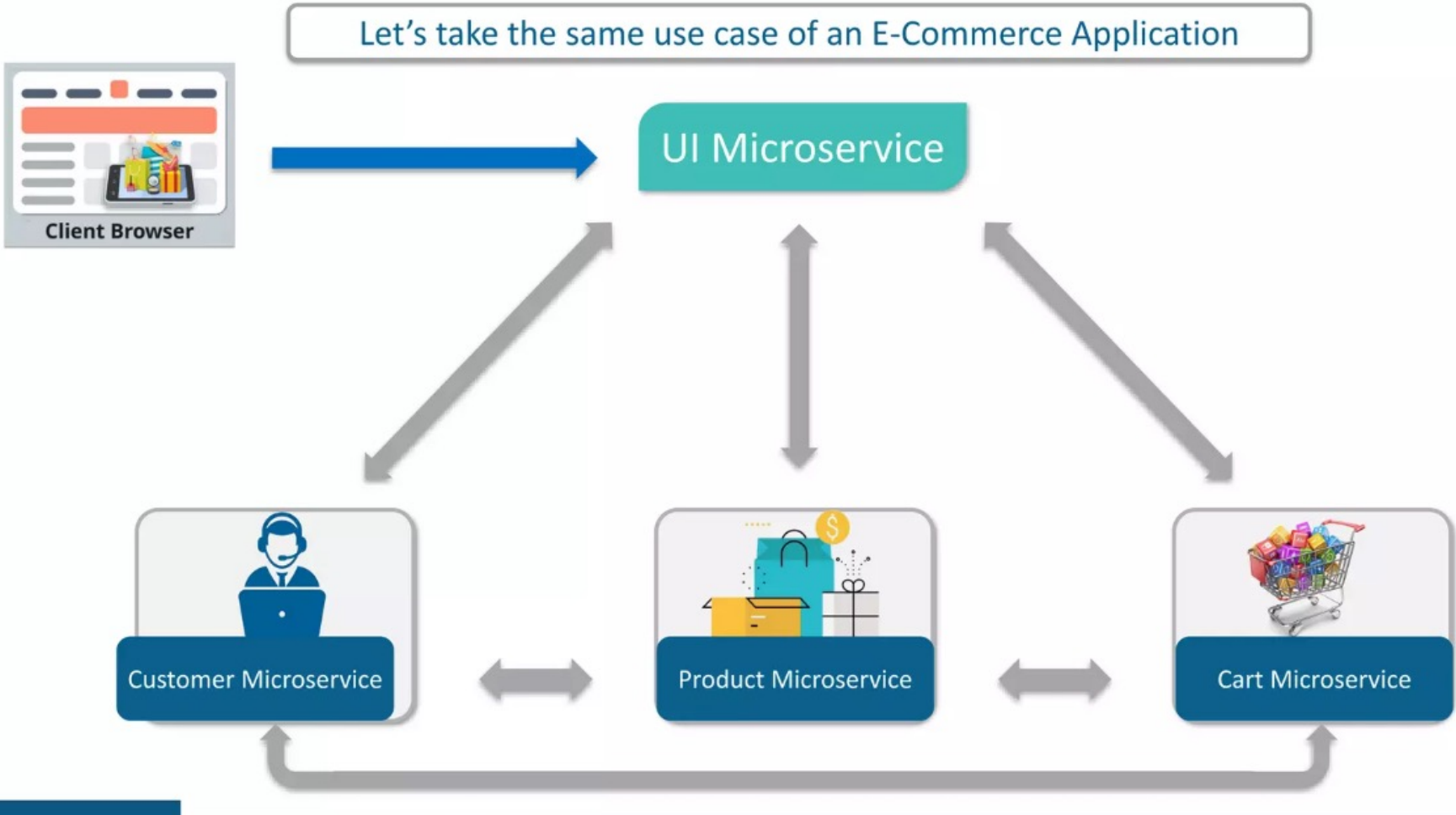
What Is Microservice Architecture?

Microservices, aka *Microservice Architecture*, is an **architectural style** that structures an application as a **collection of small autonomous services**, modelled around a **Business Domain**

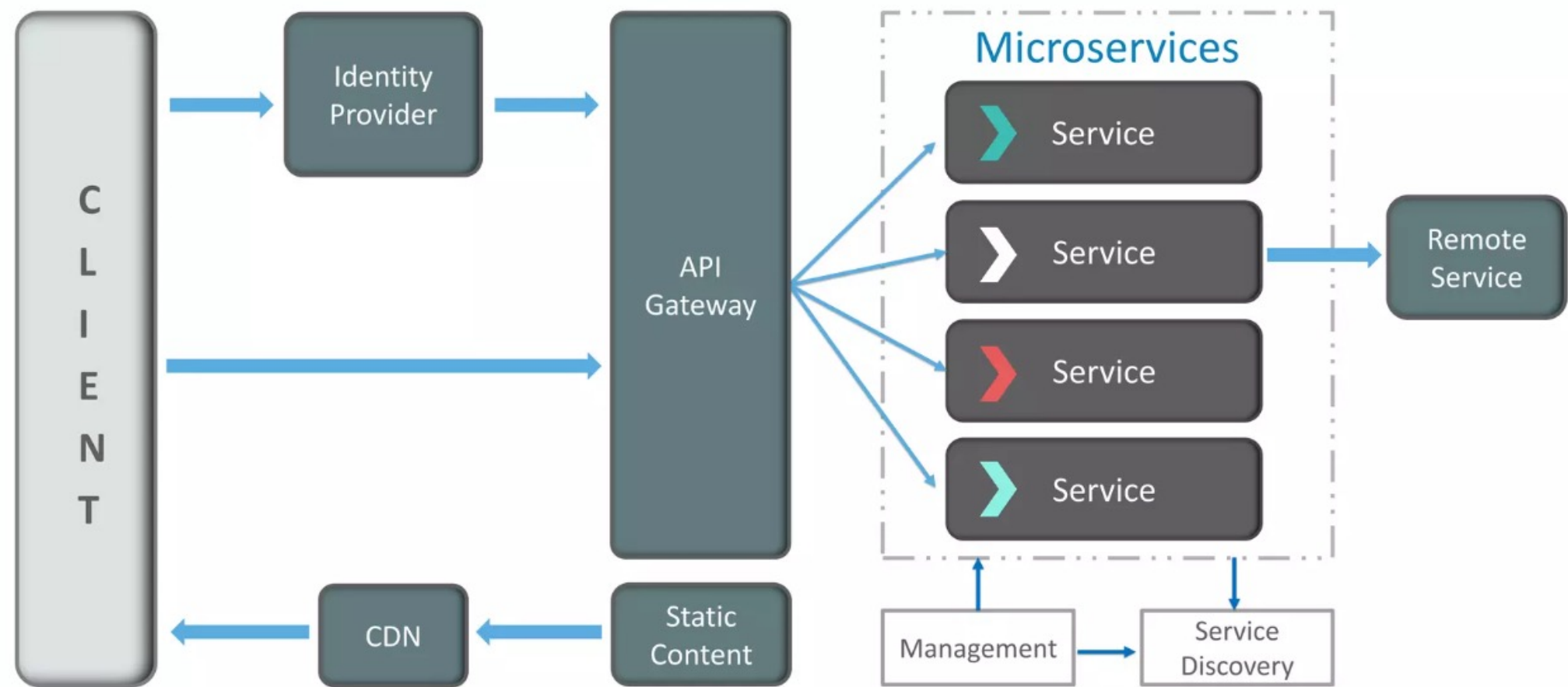


In Microservice Architecture, each service is **self-contained** and implements a **single Business capability**

Microservice Architecture - Example



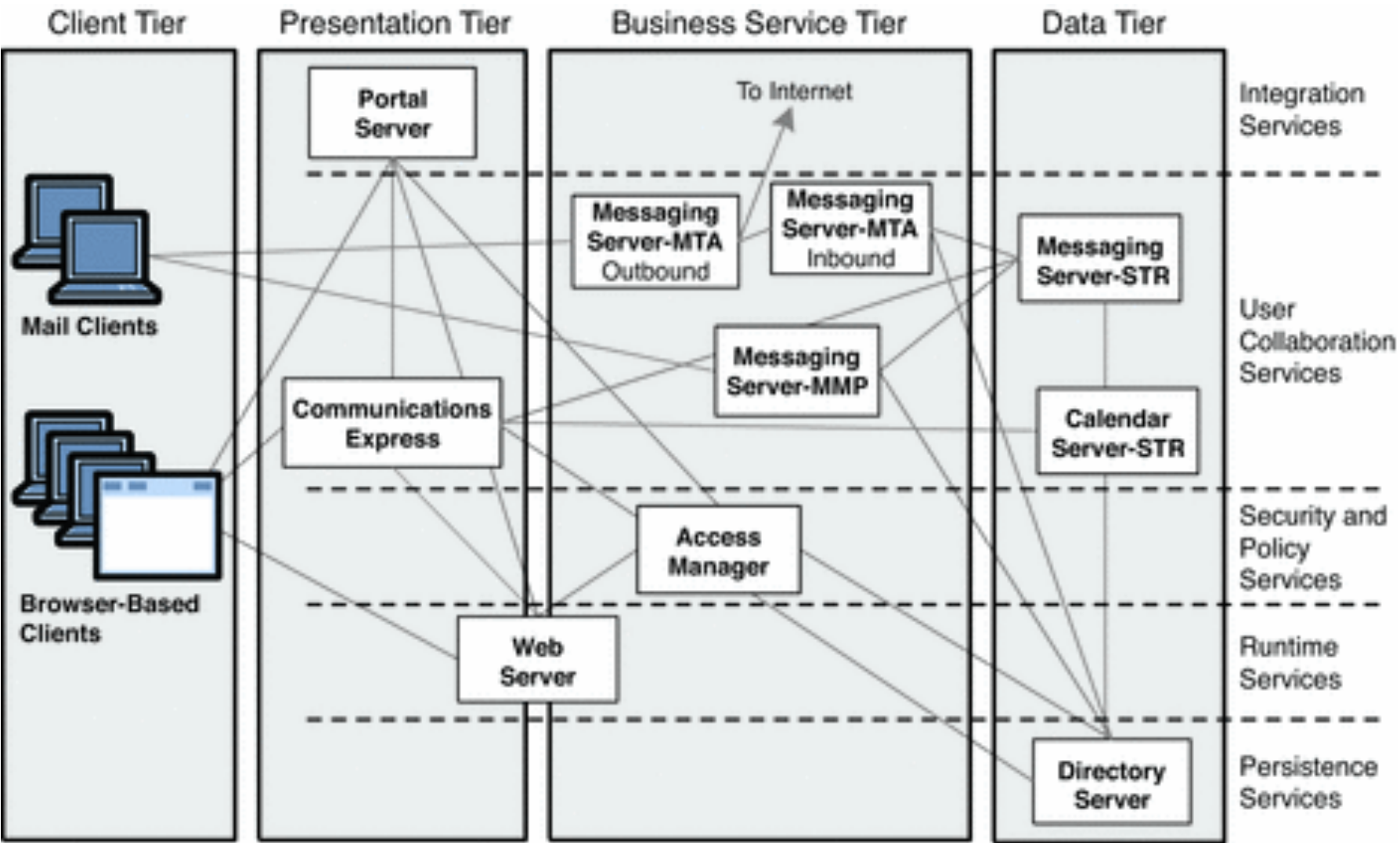
Microservice Architecture



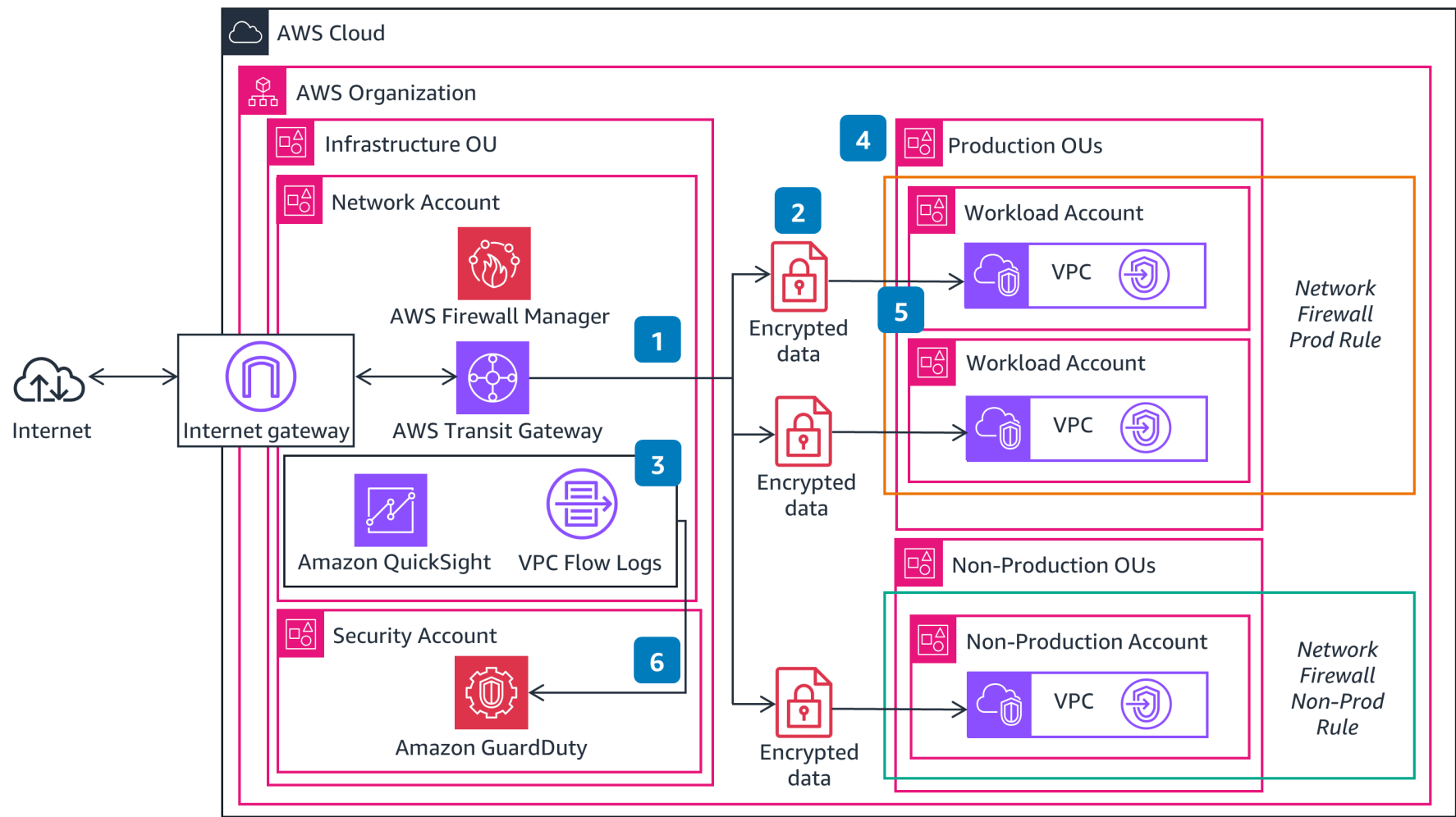
Monolithic vs Microservices

Feature	Monolithic Architecture	Microservices Architecture
Structure	Single, self-contained application	Collection of small, independent services
Deployment	Deployed as a single unit	Deployed independently
Technology Stack	Typically uses a single technology stack	Can use different technology stacks for different services
Scalability	Difficult to scale horizontally	Easier to scale horizontally by adding more instances of individual services
Development	Can be slower and more complex due to tight coupling	Faster and more agile development due to loose coupling
Testing	Can be challenging due to dependencies	Easier to test individual services
Fault Isolation	A failure in one component can affect the entire application	Failures are isolated to individual services
Continuous Delivery	Can be more difficult due to dependencies	Easier to implement continuous delivery for individual services
Team Structure	Typically a single team develops and maintains the entire application	Multiple teams can develop and maintain different services
Examples	Traditional web applications, legacy systems	Modern cloud-native applications, APIs

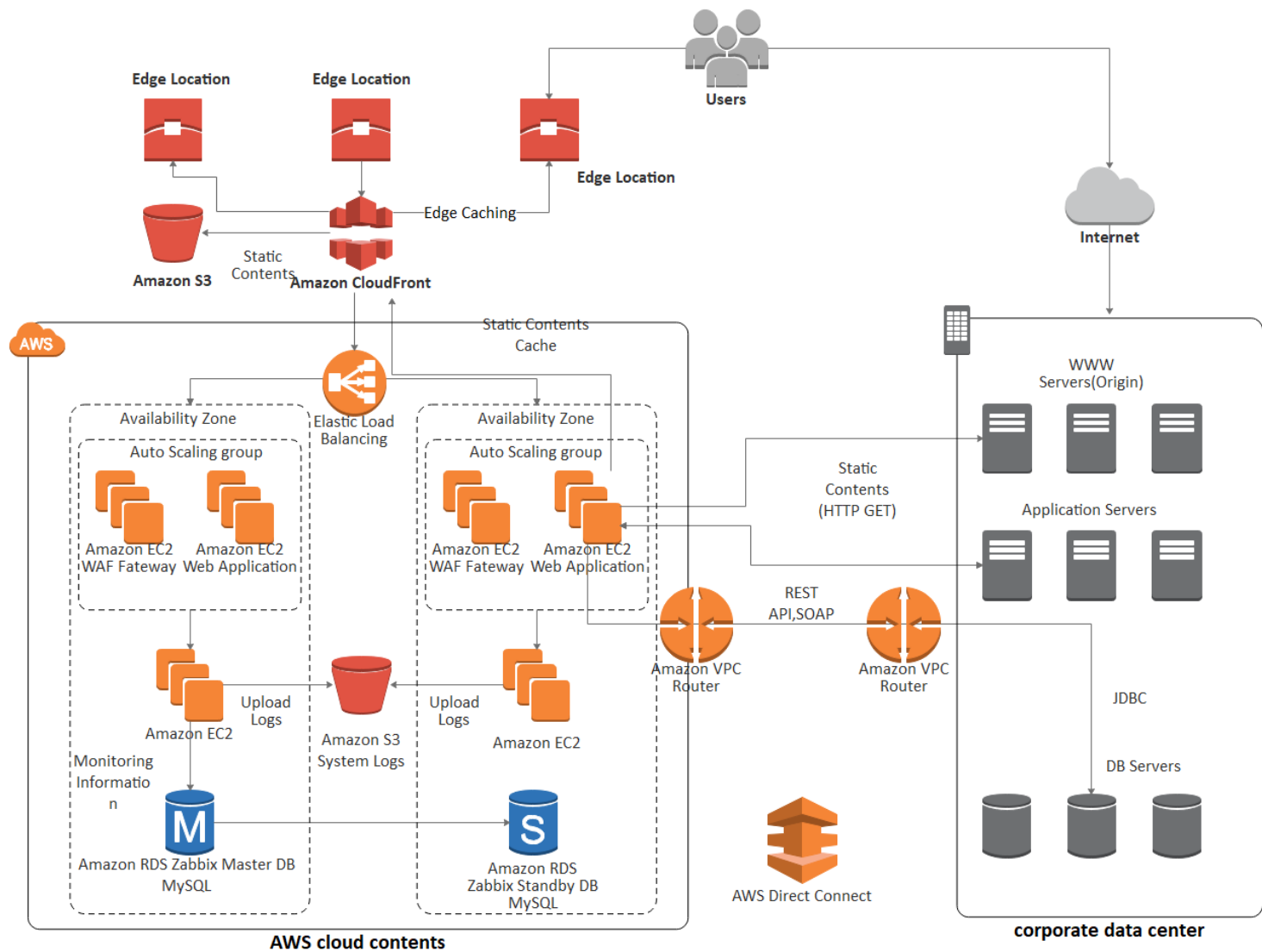
Sample – Solution Architecture



Sample – Network Architecture



Sample –Infrastructure Architecture



Sample – Security Architecture

